

Lecture 13: Designing and Building AI Agents

Architectures, tools, multi-agent systems, evaluation, and deployment

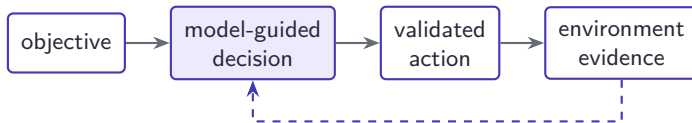
FYS4411 — Reinforcement Learning and Agentic AI
University of Oslo, Department of Physics

2026

Motivation: move from a model to a controlled system

An LLM can propose a useful next step. An operational agent must also determine what information is missing, select an admissible action, execute it through a controlled interface, interpret the result, and stop for the right reason.

The engineering problem is therefore not simply “write a better prompt.” It is to construct a system whose behavior remains useful under uncertain observations, imperfect model decisions, partial failures, changing state, and explicit resource constraints.



Motivated by Anthropic, *Building Effective Agents* (2024).

Learning goals

After this lecture, you should be able to

- ▶ choose among a single model call, deterministic workflow, single agent, and multi-agent system;
- ▶ specify agent state, actions, observations, tools, budgets, approval gates, and stopping conditions;
- ▶ implement common workflow and agent loops using typed interfaces and explicit control flow;
- ▶ design multi-agent roles, communication contracts, scheduling, and verification mechanisms;
- ▶ evaluate complete trajectories with respect to correctness, cost, latency, safety, and reproducibility.

The central design proposition

Agent engineering is interface engineering

The model should make only those decisions that benefit from semantic judgement. Conventional software should enforce invariants, permissions, schemas, budgets, logging, and irreversible-action boundaries.

Let π_θ denote the model policy and $\mathcal{A}(s)$ the actions permitted by the system in state s . The executable policy is constrained:

$$a_t \sim \pi_\theta(\cdot \mid s_t), \quad a_t \in \mathcal{A}(s_t), \quad s_{t+1} = U(s_t, a_t, o_{t+1}).$$

The quality of $\mathcal{A}$ and U is often as consequential as the quality of π_θ .

Part	Central question
I. Architecture choice	What should be agentic, and what should remain deterministic?
II. Core patterns	Which workflow or control pattern matches the task structure?
III. Single-agent engineering	How are state, tools, memory, safety, and tests implemented?
IV. Multi-agent systems	When do parallelism and specialization justify coordination?
V. Large applications	How do agents operate in research and industry?
VI. Evaluation and deployment	What evidence is required before increasing autonomy?

Part I: Choosing the right architecture

Begin with the task structure, not the label “agent”

Workflows and agents allocate control differently

Workflow

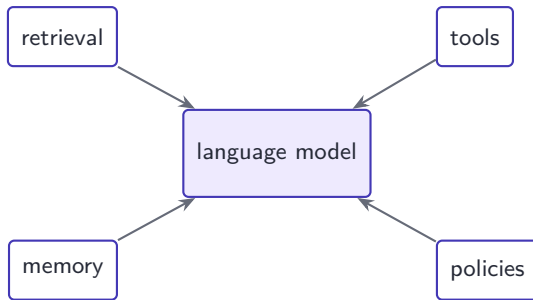
- ▶ Code specifies the execution path.
- ▶ Model calls fill bounded semantic steps.
- ▶ Branches and failure handling are known in advance.
- ▶ Predictability and auditability are relatively high.

Agent

- ▶ The model dynamically chooses steps and tools.
- ▶ The number and order of actions may be unknown.
- ▶ Environmental feedback changes the plan.
- ▶ Flexibility is obtained at additional cost and risk.

Motivated by Anthropic, *Building Effective Agents*.

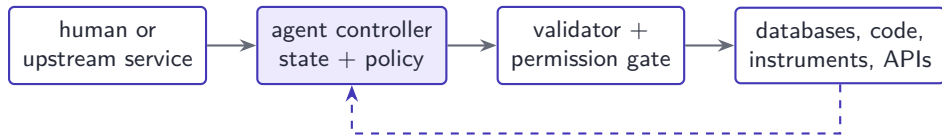
The augmented LLM is the basic component



The useful primitive is rarely a bare model. It is a model supplied with relevant context, callable capabilities, retained state, and constraints. A workflow or agent then determines how this primitive is invoked over time.

Motivated by Anthropic (2024), “Building block: the augmented LLM.”

Draw the system boundary before writing prompts



- ▶ The model proposes; the controller records and interprets.
- ▶ The gate rejects malformed, unauthorized, or unsafe actions.
- ▶ The environment supplies evidence that is not under model control.

Use agentic control only when the path is genuinely uncertain

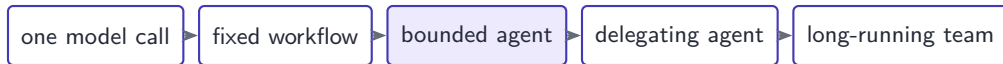
An agent is a plausible choice when several conditions hold:

- ▶ the task is open-ended and the necessary sequence of steps cannot be specified reliably in advance;
- ▶ intermediate observations should change the subsequent strategy;
- ▶ tools expose informative, timely feedback about progress;
- ▶ success, failure, and resource exhaustion can be detected;
- ▶ the value of the task justifies additional latency, cost, and operational risk.

If the path is stable, a deterministic workflow is normally easier to test, reproduce, and govern.

Motivated by Anthropic (2024), “When (and when not) to use agents.”

Autonomy is a design variable, not a binary property



Increase autonomy when

new observations require semantic replanning and the system can measure progress.

Reduce autonomy when

consequences are irreversible, feedback is weak, or a stable process can be encoded directly.

Optimize task value subject to cost and risk

A deployable agent is not selected by accuracy alone. One possible design objective is

$$\max_{\mathcal{D}} \mathbb{E}[V(y, \tau)] - \lambda_C \mathbb{E}[C(\tau)] - \lambda_L \mathbb{E}[L(\tau)] - \lambda_R \mathbb{E}[R(\tau)],$$

where $\mathcal{D}$ is the system design, τ is an execution trajectory, V is task value, and C , L , and R denote monetary cost, latency, and risk.

Implication

Adding agents, tools, or reflection loops is justified only when the measured increase in task value exceeds the coordination and execution penalties.

Motivated by Kapoor et al., *AI Agents That Matter*.

Begin with an explicit task contract

Field	Required specification
Objective	Observable outcome, not merely a topic or aspiration.
Inputs	Available data, provenance, uncertainty, and freshness.
Actions	Allowed tools, parameters, side effects, and permissions.
Evidence	Conditions that support progress, completion, or failure.
Constraints	Budget, latency, policy, privacy, and domain rules.
Escalation	Events requiring clarification, review, or human approval.
Termination	Success predicate, failure predicate, and hard limits.

Choose the least complex architecture that passes evaluation

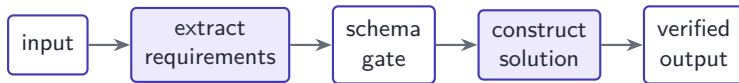
Task property	Preferred design	Primary benefit	Primary liability
Single semantic transform	one model call	low latency	limited correction
Known sequence	workflow	reproducibility	low adaptability
Unknown local path	bounded agent	dynamic replanning	compounding error
Independent subproblems	parallel workers	breadth and speed	aggregation cost
Distinct expertise or context	multi-agent system	specialization	coordination failure

Architecture is an empirical hypothesis. Evaluate it against a simpler baseline.

Part II: Core workflow and agent patterns

A design vocabulary adapted from Anthropic's production guide

Prompt chaining makes each step narrower



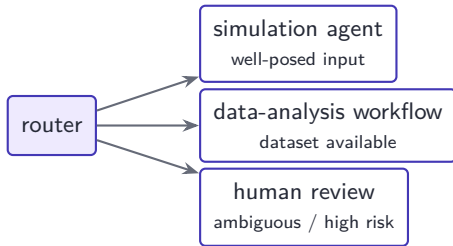
Use prompt chaining when a stable task can be decomposed into sequential transformations. Each call receives a smaller cognitive burden, and programmatic gates can reject invalid intermediate results.

Scientific example

Parse an experimental objective → validate units and bounds → produce an executable instrument configuration.

Motivated by Anthropic (2024), “Workflow: Prompt chaining.”

Routing separates qualitatively different regimes

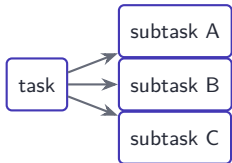


Routing is appropriate when categories are distinguishable and require different prompts, tools, models, permissions, or evaluation criteria. The router itself must be evaluated: a perfect specialist cannot repair a systematic misroute.

Motivated by Anthropic (2024), “Workflow: Routing.”

Parallelization exploits independence or diversity

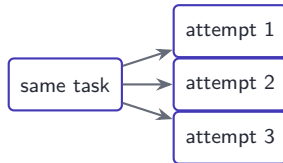
Sectioning



Independent subtasks reduce wall-clock time and isolate context.

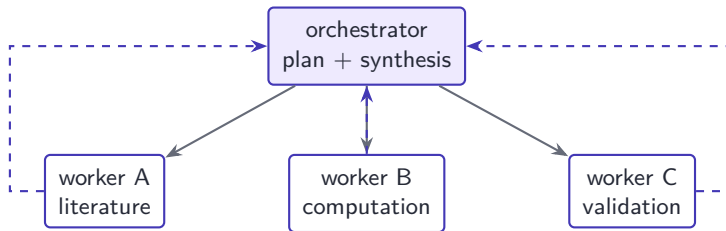
Motivated by Anthropic (2024), “Workflow: Parallelization.”

Voting or independent review



Diverse attempts improve coverage only if aggregation is reliable.

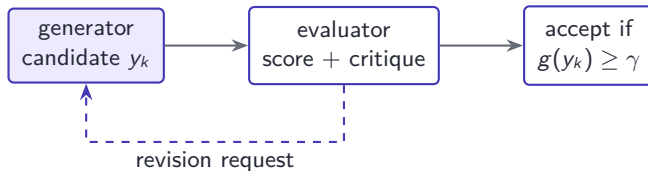
Orchestrator-workers supports dynamic decomposition



Unlike fixed parallelization, the orchestrator decides which subtasks exist for the current input. Delegations should specify objective, boundaries, tools, evidence, budget, and output schema.

Motivated by Anthropic (2024), "Workflow: Orchestrator-workers."

Evaluator–optimizer turns criteria into feedback

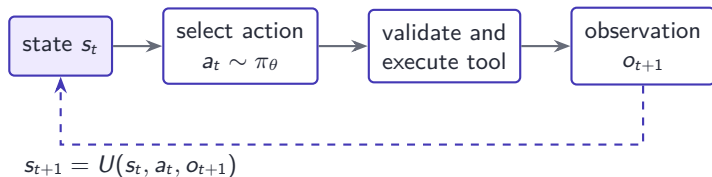


The loop is useful when evaluation criteria are sufficiently explicit and successive revisions produce measurable improvement. It is weak when the evaluator shares the generator's blind spots or when no external evidence is available.

$$y_{k+1} = G(x, y_k, E(x, y_k)), \quad \text{stop if } g(y_k) \geq \gamma.$$

Motivated by Anthropic (2024), "Workflow: Evaluator-optimizer."

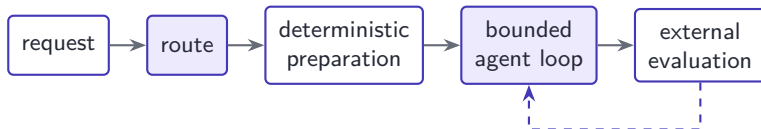
An autonomous agent closes the loop through evidence



Ground truth should enter through tool results, executable tests, measurements, or authoritative records. Model-generated self-assessment is useful diagnostic information, but it is not a substitute for environmental evidence.

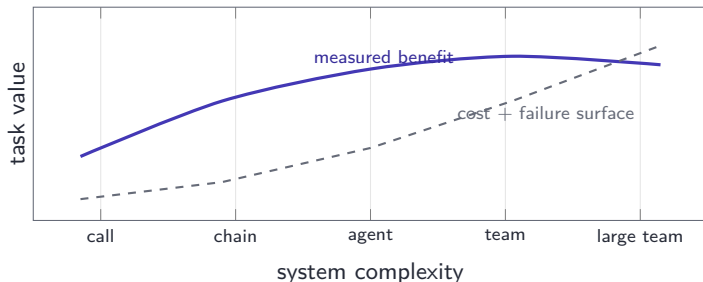
Motivated by Anthropic (2024), "Agents."

Patterns can be composed without making every step agentic



- ▶ Use code where the correct transformation is known.
- ▶ Use a model where semantic interpretation or choice is required.
- ▶ Use an agent loop only where new evidence changes the next action.
- ▶ End with a verifier that is independent of the generation path when possible.

Complexity must earn its place experimentally



The curves are conceptual. The engineering rule is empirical: compare each added component against an ablation and a simpler baseline on realistic tasks.

Motivated by Anthropic (2024); Kapoor et al. (2024).

Four recurring failure surfaces

Surface	Typical failure	Engineering response
Specification	vague objective or completion rule	task contract and acceptance tests
Interface	wrong tool, malformed arguments, excessive output	typed schemas and tool-level evals
Control	loops, premature stopping, poor replanning	budgets, progress tests, explicit states
Coordination	duplicated work, information loss, conflict	role boundaries and message contracts

Most failures attributed to “the model” are interactions between the model and one of these surfaces.

Part III: Engineering a reliable single agent

State, tools, control, safety, observability, and tests

The unit of behavior is a trajectory

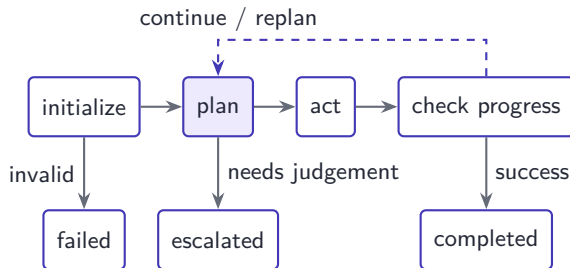
Define the trajectory

$$\tau = (s_0, a_0, o_1, s_1, a_1, o_2, \dots, s_T),$$

where s_t is controlled state, a_t is an attempted action, and o_{t+1} is the resulting observation. A complete implementation should make each element inspectable.

Object	Examples
s_t	task contract, plan, evidence ledger, budgets, permissions, open questions
a_t	tool call, delegation, clarification request, final answer, stop
o_{t+1}	API response, test result, measurement, timeout, validation error

Represent control flow as explicit states



Explicit states prevent informal prompt text from becoming the only description of lifecycle behavior.

A tool is a contract across a stochastic boundary

The caller is non-deterministic; the tool implementation should be as deterministic and explicit as possible.

Input contract

- ▶ precise name and purpose;
- ▶ typed fields, units, ranges;
- ▶ mutually exclusive options;
- ▶ permission and side-effect class.

Output contract

- ▶ status and machine-readable result;
- ▶ provenance and timestamps;
- ▶ recoverable versus terminal errors;
- ▶ concise evidence for the next decision.

Motivated by Anthropic, *Writing Effective Tools for Agents* (2025).

Code: define a narrow, typed tool schema

```
run_simulation = {
  "name": "run_simulation",
  "description": "Run one validated Ising-model experiment.",
  "input_schema": {
    "type": "object",
    "properties": {
      "temperature_K": {"type": "number",
                        "minimum": 0.01, "maximum": 20.0},
      "lattice_size": {"type": "integer",
                       "enum": [16, 32, 64, 128]},
      "seed": {"type": "integer", "minimum": 0}
    },
    "required": ["temperature_K", "lattice_size", "seed"],
    "additionalProperties": False
  },
  "side_effect": "compute-only"
}
```

The schema removes invalid regions before execution; it does not ask the model to remember every constraint.

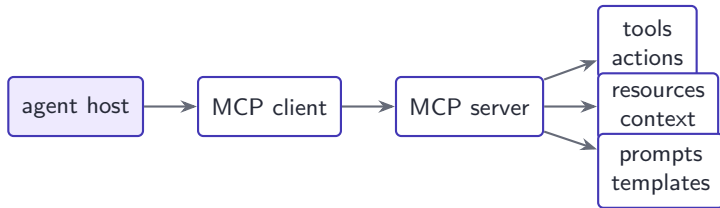
Design tools around decisions, not database structure

Weak interface	Stronger interface
<code>query_table(sql)</code> with broad credentials	<code>get_validated_samples(filters)</code> with read-only scope
Return a 40,000-token raw log	return summary, anomalies, and a retrievable artifact id
<code>update_record(payload)</code>	<code>propose_update(...)</code> then approved <code>commit_update(id)</code>
Many overlapping tool names	namespaced, mutually distinct capabilities

Good tools enlarge the set of successful strategies while narrowing the set of harmful or ambiguous actions.

Motivated by Anthropic (2025), *Writing Effective Tools for Agents*.

MCP standardizes the integration boundary



The Model Context Protocol provides a standard way to discover and invoke external capabilities. Standardization reduces integration duplication; it does not remove the need for authentication, authorization, validation, or domain-specific tool evaluation.

Motivated by Model Context Protocol documentation.

The minimal agent loop has five control decisions

1. **Assemble context**: task contract, relevant state, evidence, and available actions.
2. **Propose**: request either a typed action, clarification, final result, or stop.
3. **Validate**: enforce schema, permissions, budgets, and state-dependent invariants.
4. **Execute**: call the tool in an isolated runtime and normalize its observation.
5. **Update**: append evidence, test progress, and either continue, replan, escalate, or terminate.

Separation of responsibilities

The model selects among admissible semantic options. The controller owns the lifecycle.

Code: a framework-independent bounded loop

```
def run_agent(task, tools, budget):
    state = initialize(task, budget)
    while not terminal(state):
        context = build_context(state, tools)
        proposal = model.decide(context, response_schema=Decision)

        decision = validate(proposal, state, tools)
        if decision.kind == "ask_human":
            return checkpoint(state, decision.question)
        if decision.kind == "finish":
            return verify_and_finalize(state, decision.answer)

        observation = execute_safely(
            tools[decision.tool], decision.arguments, state.permissions)
        state = update_state(state, decision, normalize(observation))

    return terminal_result(state)
```

Normalize observations before returning them to the model

Tool outputs are part of the control interface. A normalized observation can be written as

$$o_{t+1} = \{\text{status, summary, evidence, artifact, uncertainty, retryability}\}.$$

- ▶ Separate data from instructions embedded in untrusted content.
- ▶ Preserve provenance and exact artifacts outside the context window.
- ▶ Return compact decision-relevant evidence, not arbitrary payloads.
- ▶ Distinguish an empty result from an execution failure.

This layer is also the natural location for redaction, unit conversion, and error taxonomy.

Planning should expose assumptions and revision triggers

A useful plan is not a long narrative. It is a testable data structure:

$$P_t = \{(g_i, \text{dependency}_i, \text{evidence}_i, \text{status}_i, \text{budget}_i)\}_{i=1}^m.$$

Replan when

- ▶ a dependency fails;
- ▶ evidence contradicts an assumption;
- ▶ expected information gain is low;
- ▶ a budget threshold is crossed.

Do not replan when

- ▶ the next action is already valid;
- ▶ only wording has changed;
- ▶ reflection supplies no new evidence;
- ▶ a deterministic retry policy applies.

Memory has several distinct functions

Memory type	Timescale	Examples
Working state	one execution	plan, budgets, current evidence, pending calls
Episodic	across tasks	prior trajectories, failures, resolved exceptions
Semantic	durable	approved facts, manuals, domain knowledge, policies
Procedural	versioned	tool instructions, strategies, templates, skills

Write policies must be stricter than read policies. Persisted mistakes can influence many later executions.

Context is a scarce control surface

At step t , context selection may be formulated as a budgeted relevance problem:

$$\max_{S \subseteq \mathcal{M}} I(S; a_t^*) \quad \text{subject to} \quad \sum_{m \in S} \text{tokens}(m) \leq B_t.$$

- ▶ Keep the active objective, constraints, and recent evidence explicit.
- ▶ Retrieve older material on demand rather than copying the entire history.
- ▶ Compact completed phases into factual summaries with artifact references.
- ▶ Record critical decisions outside the model context in durable state.

Motivated by Anthropic, *Effective Context Engineering for AI Agents* (2025).

Structured decisions make invalid behavior observable

Instead of accepting unrestricted prose, require one of a finite set of decision variants:

$$d_t \in \{\text{CallTool}, \text{AskHuman}, \text{Finish}, \text{Stop}\}.$$

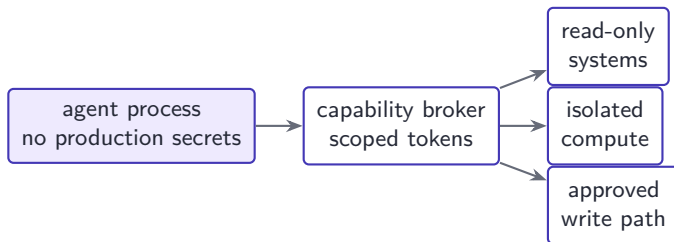
Validation layer	Question
Syntax	Does the output conform to the declared schema?
Semantic	Are units, identifiers, and dependencies meaningful?
State-dependent	Is this action permitted now?
Policy	Does the action require review or prohibition?

Approval gates should follow consequence, not uncertainty alone

impact	human approval before action	two-person or policy review
	automatic read / simulate	automatic with logging
reversibility		

Approval should be attached to the proposed action and its computed diff, not to a vague conversation checkpoint.

Sandboxing separates reasoning from authority



- ▶ Issue short-lived credentials with least privilege.
- ▶ Separate code generation from code execution.
- ▶ Restrict network, filesystem, process, and data access independently.
- ▶ Preserve immutable logs for privileged operations.

Retries require idempotency and error classification

For action a with idempotency key k , a safe executor should ensure

$$\text{execute}(a, k); \text{execute}(a, k) \equiv \text{execute}(a, k).$$

Error class	Example	Response
Transient	rate limit, network timeout	bounded exponential backoff
Correctable	invalid field, missing unit	return structured correction
State conflict	version changed	refresh state and replan
Terminal	permission denied, unsafe request	stop or escalate

Stopping rules are part of correctness

Terminate when any explicit predicate is satisfied:

$$\text{stop}(s_t) = \text{success}(s_t) \vee \text{fatal}(s_t) \vee (C_t > C_{\max}) \vee (T_t > T_{\max}) \vee \text{stalled}(s_t).$$

- ▶ **Success:** acceptance tests pass and required evidence exists.
- ▶ **Fatal:** an invariant, permission, or dependency cannot be satisfied.
- ▶ **Budget:** cost, wall time, tokens, or tool calls exceed a limit.
- ▶ **Stalled:** repeated actions fail to increase measurable progress.

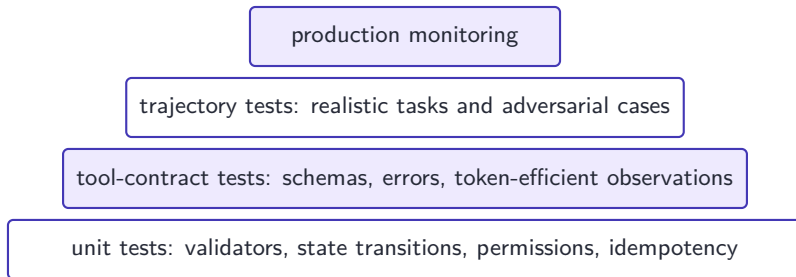
“The model says it is done” is a proposal to test, not a completion predicate.

Tracing should reconstruct every consequential decision

Trace field	Purpose
Execution metadata	task version, model, prompt, tool and policy versions
Decision record	selected action, alternatives if available, validation result
Tool span	arguments, permission, latency, status, observation digest
State transition	evidence added, budget consumed, plan status changed
Outcome	termination reason, grader results, human interventions

A trace is both an operational instrument and the primary data structure for agent evaluation.

Test the system from deterministic components upward



Deterministic tests should be numerous and fast. Stochastic trajectory tests require repeated trials, confidence intervals, and artifact isolation. Production failures should become regression tasks.

Motivated by Anthropic, *Demystifying Evals for AI Agents* (2026).

Part IV: Designing multi-agent systems

Specialization and parallelism under explicit coordination

Multiple agents are useful for four structural reasons

Reason	System mechanism	Example
Parallelism	independent trajectories execute concurrently	breadth-first literature or market research
Specialization	distinct prompts, tools, models, or permissions	planner, simulator, verifier, compliance reviewer
Context separation	each worker receives only relevant material	different code modules or data modalities
Independent checking	separate paths reduce correlated omission	calculation, safety, or citation review

The design must state which of these mechanisms is expected to improve the task. “More agents” is not itself a mechanism.

Do not use multiple agents for tightly sequential reasoning

Multi-agent coordination is usually a poor fit when

- ▶ every step depends strongly on the complete result of the preceding step;
- ▶ agents require the same large context and cannot partition it meaningfully;
- ▶ the task is inexpensive enough that duplicated token use dominates value;
- ▶ no reliable aggregator can resolve conflicting or incomplete outputs;
- ▶ the single-agent baseline already satisfies the acceptance criteria.

Anthropic reports that multi-agent research is strongest for breadth-first queries, but can use roughly $15\times$ the tokens of ordinary chat and performs poorly when dependencies dominate.

Motivated by Anthropic, *How We Built Our Multi-Agent Research System* (2025).

Specify a team as roles plus interfaces

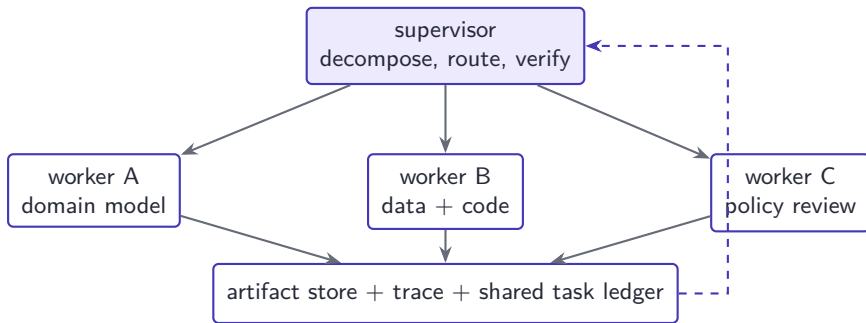
For each agent i , define a role contract

$$\mathcal{R}_i = (G_i, \mathcal{O}_i, \mathcal{A}_i, \mathcal{M}_i, B_i, E_i, T_i),$$

where G_i is its objective; $\mathcal{O}_i$ observations; $\mathcal{A}_i$ actions; $\mathcal{M}_i$ message schema; B_i budget; E_i evaluation; and T_i termination.

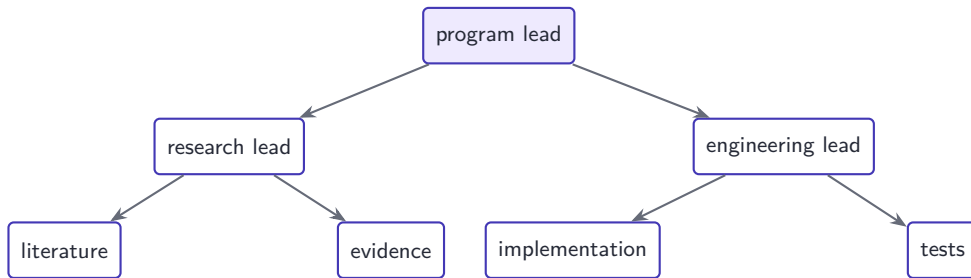
- ▶ Objectives should be jointly sufficient and minimally overlapping.
- ▶ Permissions should follow role, not inherit from the supervisor.
- ▶ Every delegation should contain boundaries and an expected artifact.
- ▶ The orchestrator must know how to detect gaps and duplication.

Supervisor–worker is the common production topology



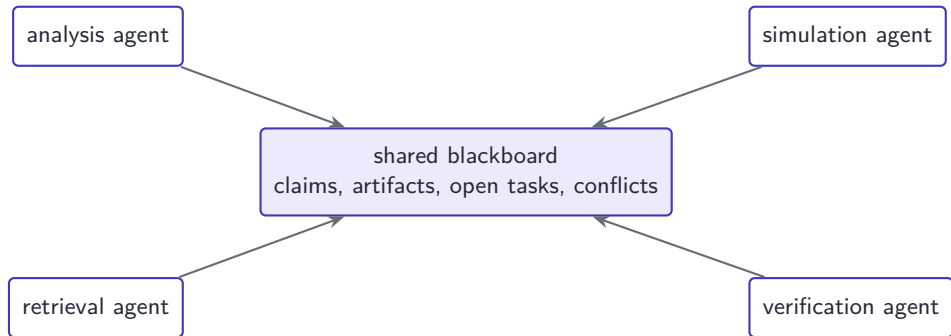
Centralized supervision simplifies user interaction and policy enforcement, but the supervisor becomes a bottleneck and a single point of synthesis failure.

Hierarchical teams control a large decomposition



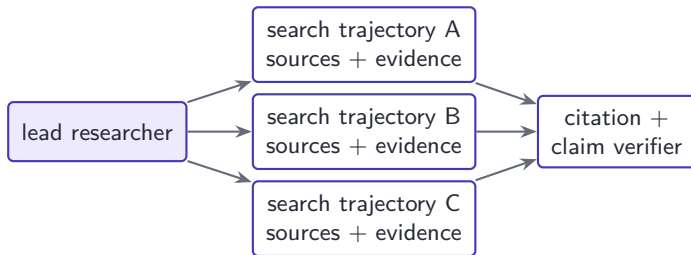
Hierarchy reduces the supervisor's fan-out and supports local expertise. It also increases the chance that evidence is compressed or distorted at each level; summaries should retain artifact references and confidence.

A blackboard supports asynchronous collaboration



Agents post structured contributions to shared state rather than sending unconstrained conversational messages. Versioning, ownership, conflict resolution, and provenance become database concerns.

Parallel research agents act as context compressors



Each worker explores a distinct information path in its own context window and returns a compact evidence package. This gains breadth only when the lead assigns non-overlapping scopes and the verifier checks claim–source alignment.

Motivated by Anthropic (2025), multi-agent Research architecture.

Debate and critique are verification patterns, not guarantees



Independent proposals can expose alternative hypotheses. However, if all agents share the same model, context, and missing evidence, errors remain correlated. The final judge should use executable checks, source evidence, or a domain model whenever possible.

$$\text{Var}\left(\frac{1}{N} \sum_i \epsilon_i\right) = \frac{\sigma^2}{N} [1 + (N-1)\rho],$$

so positive error correlation ρ limits the benefit of voting.

Messages should transfer evidence, not hidden reasoning

Message field	Content
task_id, parent_id	traceable relationship to the decomposition
objective, scope	bounded question and explicit exclusions
status	complete, partial, blocked, or failed
claims []	concise conclusions with confidence and qualifiers
evidence []	source or artifact identifiers and exact locations
open_issues []	missing data, contradictions, recommended follow-up
usage	tokens, time, tool calls, and cost

Code: supervise parallel workers with typed outputs

```
async def run_team(question, worker_specs, budget):
    tasks = [
        run_worker(Delegation(
            objective=spec.objective(question),
            scope=spec.scope,
            tools=spec.allowed_tools,
            budget=budget.allocate(spec.weight),
            output_schema=EvidencePackage))
        for spec in worker_specs
    ]
    packages = await asyncio.gather(*tasks, return_exceptions=True)
    valid = [validate_package(p) for p in packages if not failed(p)]
    coverage = assess_coverage(question, valid)
    if coverage.gaps:
        valid += await fill_gaps(coverage.gaps, budget.remaining)
    return verify_and_synthesize(question, valid)
```

Parallel execution does not eliminate orchestration: gaps, failures, duplication, and synthesis remain explicit program logic.

Allocate budget according to expected information gain

For candidate delegation j , define an approximate priority

$$q_j = \frac{\mathbb{E}[\Delta V_j \mid s_t]}{\mathbb{E}[C_j] + \alpha \mathbb{E}[L_j] + \beta \mathbb{E}[R_j]}.$$

- ▶ Start with a small exploratory allocation.
- ▶ Expand workers whose evidence changes the synthesis or resolves a gap.
- ▶ Cancel redundant or stalled trajectories.
- ▶ Reserve budget for independent verification and final integration.

The supervisor needs budget state as an input; otherwise delegation becomes an unconstrained language-generation problem.

Coordination has a measurable break-even point

Let V_N be the expected task value of N agents and $C_{\text{coord}}(N)$ the cost of delegation, communication, conflict resolution, and synthesis. Use multiple agents only if

$$V_N - V_1 > C_{\text{coord}}(N) + C_{\text{extra}}(N).$$

For a task graph $G = (V, E)$, the achievable speedup is bounded by dependencies. If fraction f is inherently sequential, Amdahl's law gives

$$S(N) \leq \frac{1}{f + (1 - f)/N}.$$

More workers cannot accelerate the sequential portion; they may increase communication load around it.

Multi-agent failures cluster into three categories

Category	Failure examples	Design response
Specification and design	ambiguous roles, missing capabilities, wrong topology	role tests, architecture ablations
Inter-agent alignment	duplicated work, information loss, conflicting assumptions	message schemas, shared ledger, gap checks
Verification and termination	false completion, circular review, no conflict resolution	external graders, stop predicates, arbiter

A study of five multi-agent frameworks across more than 150 tasks identified 14 fine-grained failure modes within these categories.

Motivated by Cemri et al., *Why Do Multi-Agent LLM Systems Fail?* (2025).

Sequential dependencies compound unreliability

If a workflow requires K dependent stages and stage k succeeds with probability p_k , a simple approximation is

$$P(\text{end-to-end success}) \approx \prod_{k=1}^K p_k.$$

Even with $p_k = 0.95$, ten indispensable stages give $0.95^{10} \approx 0.60$.

Reduce the chain

- ▶ remove non-essential calls;
- ▶ combine deterministic transforms;
- ▶ parallelize independent evidence.

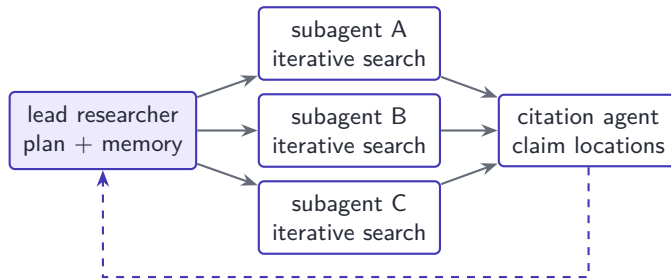
Strengthen the chain

- ▶ validate at interfaces;
- ▶ retry only recoverable failures;
- ▶ verify critical transitions externally.

Part V: Large-scale applications

Agents as operational systems for research and industry

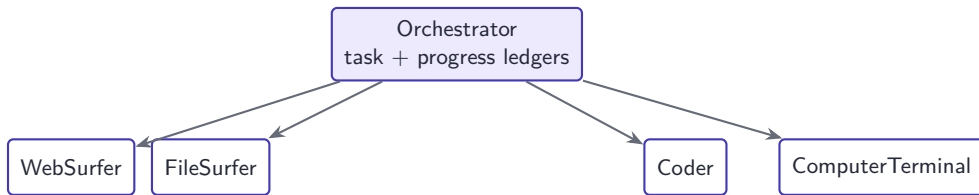
Case study: Anthropic's multi-agent Research system



Anthropic reports a 90.2% improvement over a single-agent baseline on an internal breadth-first research evaluation, but approximately $15\times$ chat token usage. The case demonstrates both the benefit and the economic constraint of parallel context windows.

Motivated by Anthropic, *How We Built Our Multi-Agent Research System* (2025).

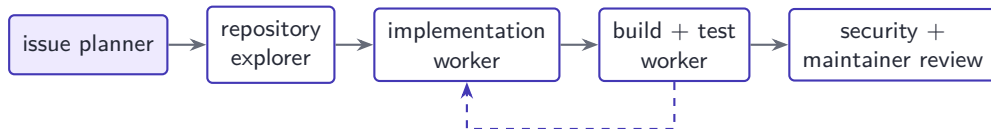
Case study: Magentic-One separates operational capabilities



Microsoft's Magentic-One uses a lead orchestrator with an outer task ledger and inner progress loop. Specialized agents browse, inspect files, write code, and execute it. The separation makes capabilities replaceable and permission boundaries clearer, while the orchestrator retains responsibility for progress and completion.

Motivated by Microsoft Research, *Magentic-One* (2024).

Software engineering agents operate against executable evidence

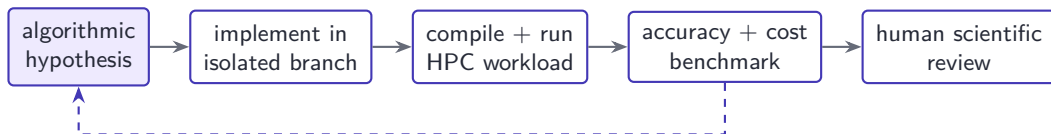


The essential feedback is not stylistic agreement; it is compilation, unit tests, integration tests, static analysis, performance measurement, and human review of architecture and security. Parallel work requires module ownership and merge discipline.

Anthropic's 2026 C-compiler stress test used 16 agents across nearly 2,000 sessions, illustrating scale as well as substantial cost and integration effort.

Motivated by Anthropic, *Building a C Compiler with a Team of Parallel Claudes* (2026).

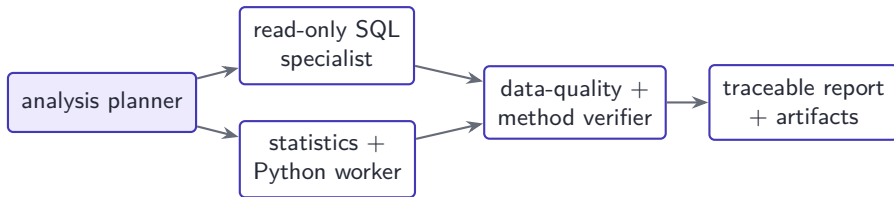
Scientific computing agents can manage long evidence loops



Suitable tasks include porting numerical solvers, debugging against a reference implementation, searching algorithm spaces, and optimizing kernels. The model proposes code; scientific acceptance remains tied to convergence, invariants, reference solutions, and independent replication.

Motivated by Anthropic, *Long-running Claude for Scientific Computing* (2026); Google DeepMind, AlphaEvolve.

Data and analytics systems separate query, computation, and review



Production controls include row-level access, query budgets, snapshot identifiers, reproducible code, statistical assumptions, and lineage from each reported quantity to a query or artifact. Natural-language fluency is not a substitute for data provenance.

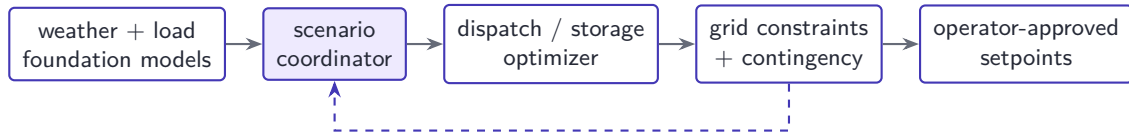
Industrial operations combine models with optimization and policy

Role	Capability	Evidence	Boundary
Demand agent	forecasts and scenario assumptions	backtests, uncertainty	cannot place orders
Planning agent	MILP / scheduling tools	feasible schedule, shadow prices	capacity constraints
Procurement agent	supplier and contract analysis	approved vendor records	spend limits
Risk agent	resilience and compliance checks	policy rules, stress tests	veto / escalate

The LLM coordinates heterogeneous evidence; domain solvers establish feasibility. A human approves material commitments and policy exceptions.

Motivated by AWS, Multi-agent collaboration guidance.

Energy systems require a model–optimization–control separation



Agents can assemble scenarios, invoke forecasters and optimization tools, explain trade-offs, and investigate anomalies. Safety-critical control remains protected by deterministic grid constraints, certified controllers, and operator authority.

Healthcare agents should coordinate evidence, not replace accountability

Research and operations

- ▶ cohort construction and trial feasibility;
- ▶ multimodal evidence synthesis;
- ▶ protocol consistency checks;
- ▶ scheduling and administrative workflows.

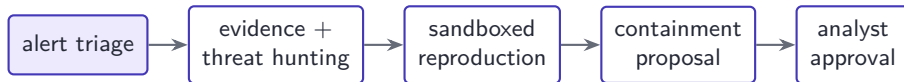
The correct deployment target is often decision support with explicit review, not autonomous diagnosis or treatment.

Required controls

- ▶ approved data use and access logging;
- ▶ calibrated uncertainty and subgroup analysis;
- ▶ source-level evidence and versioned guidelines;
- ▶ qualified human decision-maker.

Motivated by Google Cloud, Agentic AI framework in life sciences for R&D (2025).

Cybersecurity agents can increase both defense and attack scale



Agents can correlate telemetry, inspect code, and test hypotheses in isolated environments. Network blocking, credential rotation, and production changes should require explicit policy and approval. The same long-horizon tool use can also automate vulnerability discovery, so capability evaluation must include misuse risk.

Motivated by Anthropic, *AI Agents Find Smart Contract Exploits* (2025).

Enterprise systems must preserve ownership and policy

Agent	Contribution	Authoritative boundary
Knowledge agent	retrieves internal evidence	source ACLs and document versions
Finance agent	computes exposure and scenarios	governed financial systems
Legal agent	identifies clauses and obligations	counsel-approved interpretations
Compliance agent	checks policy and jurisdiction	versioned rule engine
Executive synthesizer	integrates evidence and uncertainty	accountable human owner

Cross-domain synthesis is valuable precisely because ownership remains distributed. The agent should not silently collapse disagreement between authorities.

Motivated by AWS, Multi-agent business expert for biopharmaceutical analysis (2025).

A production reference architecture separates six planes

experience plane: user intent, status, explanation, approval

control plane: routing, state machine, budgets, scheduler, stopping

agent plane: model policies, roles, context assembly, delegation

capability plane: typed tools, domain models, solvers, MCP servers

trust plane: identity, permissions, sandbox, policy, secrets

evidence plane: traces, artifacts, evaluations, monitoring, audit

The model belongs in the agent plane. Production reliability emerges from the interaction of all six planes.

Part VI: Evaluation and deployment

Increase autonomy only when trajectory-level evidence supports it

Evaluate the complete trajectory, not the final prose alone

An agent may produce a plausible final answer after unsafe, wasteful, or irreproducible actions. Evaluation should therefore score

$$G(\tau) = w_o G_{\text{outcome}} + w_p G_{\text{process}} + w_e G_{\text{evidence}} - w_c C - w_r R.$$

Level	Evaluation question
Outcome	Was the externally defined task completed?
Process	Were actions valid, efficient, and policy-compliant?
Evidence	Are claims supported by traceable observations or artifacts?
Recovery	Did the system respond correctly to injected failures?

A useful metric suite is multi-objective

Dimension	Metric examples	Why it matters
Correctness	task success, test pass rate, factual precision	primary external value
Efficiency	tokens, tool calls, compute, wall time, cost	economic and environmental viability
Robustness	variance, recovery rate, perturbation sensitivity	reliability under ordinary failures
Safety	policy violations, unauthorized attempts, severity	consequence control
Human factors	intervention rate, review time, calibrated trust	operational fit

Report a Pareto frontier; do not compress all objectives into one unexplained score.

Motivated by Kapoor et al., *AI Agents That Matter* (2024).

Combine deterministic, model-based, and human graders

Grader	Strength	Limitation
Executable	exact tests, schemas, constraints, numerical tolerances	covers only encoded criteria
Reference-based	compares against approved outputs or artifacts	reference construction can be expensive
Model-based	scalable semantic rubric and error classification	bias, drift, prompt sensitivity
Human expert	domain judgement and consequence awareness	costly and variable

Use executable evidence first, then model graders for triage and human experts for consequential or ambiguous cases.

Motivated by Anthropic, *Demystifying Evals for AI Agents* (2026).

Offline evaluation and online monitoring answer different questions

Offline before deployment

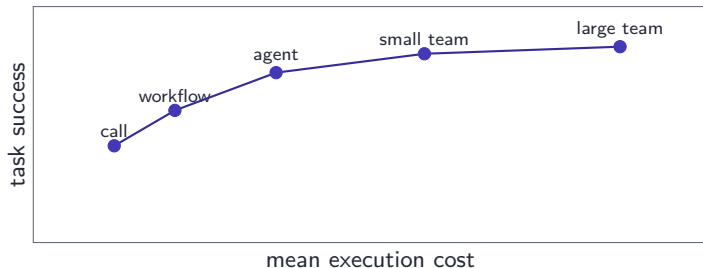
- ▶ representative and adversarial tasks;
- ▶ fixed tool and data snapshots;
- ▶ repeated trials with uncertainty;
- ▶ architecture ablations;
- ▶ injected timeouts and malformed outputs.

Production failures should be sanitized, labelled, and added to the offline regression suite.

Online after deployment

- ▶ success and escalation rates;
- ▶ drift in tool use and cost;
- ▶ policy attempts and blocked actions;
- ▶ human overrides and corrections;
- ▶ sampled trace review.

Select architectures on the cost–quality frontier



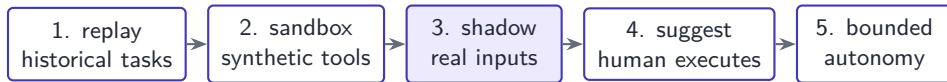
Choose the lowest-cost design that clears the required quality and risk thresholds. Average success without cost can reward unnecessarily elaborate systems.

Security analysis follows the entire action path

Threat	Mechanism	Control
Prompt injection	untrusted content supplies malicious instructions	content separation, least privilege, approval
Data exfiltration	tools or outputs expose protected information	scoped access, egress filters, redaction
Confused deputy	agent uses legitimate authority for wrong purpose	intent binding, state-dependent authorization
Supply chain	compromised tool, model, dependency, or prompt	pinning, signing, isolation, provenance
Runaway execution	recursive delegation or repeated actions	depth, budget, concurrency, and time limits

Motivated by Anthropic, Prompt-injection defenses for browser use (2025).

Deployment should increase consequence gradually



Promotion requires predefined evidence: stable success, acceptable tail cost, low severe-failure rate, effective recovery, and successful human review. Rollback must be technically immediate and operationally rehearsed.

Governance principle

Autonomy is granted to a versioned system for a bounded task and permission set; it is not granted to a model in the abstract.

Build sequence: a practical checklist

1. Define a task contract and collect representative trajectories.
2. Establish a one-call or deterministic-workflow baseline.
3. Add typed tools with narrow permissions and contract tests.
4. Implement explicit state, budgets, stopping, tracing, and recovery.
5. Add an agent loop only for decisions that require dynamic replanning.
6. Add workers only for measured parallelism, specialization, or verification.
7. Evaluate outcome, process, cost, robustness, and safety with ablations.
8. Deploy through sandbox, shadow, approval, and bounded-autonomy stages.

Every failure that matters should become either an invariant, a test, a grader, or an approval boundary.

Closing synthesis

What makes an agent useful

Useful agents connect model judgement to well-designed tools, explicit state, environmental evidence, and controlled action.

What makes a multi-agent system useful

Useful teams exploit independent work, specialized context, or independent verification while making coordination and synthesis observable.

What makes either system deployable

Deployable systems have bounded authority, reproducible traces, trajectory-level evaluation, tested failure recovery, and accountable human ownership.

The objective is not maximum autonomy. It is maximum defensible task value.

Selected references and implementation guides

- ▶ Anthropic, Building Effective Agents (2024).
- ▶ Anthropic, How We Built Our Multi-Agent Research System (2025).
- ▶ Anthropic, Writing Effective Tools for Agents (2025).
- ▶ Anthropic, Demystifying Evals for AI Agents (2026).
- ▶ Model Context Protocol documentation.
- ▶ Microsoft Research, Magentic-One (2024).
- ▶ Kapoor et al., AI Agents That Matter (2024).
- ▶ Cemri et al., Why Do Multi-Agent LLM Systems Fail? (2025).
- ▶ AWS, Multi-agent collaboration pattern.
- ▶ Anthropic, Long-running Claude for Scientific Computing (2026).

Links are embedded throughout the lecture; publication dates reflect the cited versions.